# Process of Solving issue #12984 for Kestra

## What is the open source project about?

Kestra is an open-source workflow orchestration platform that helps engineers automate and manage complex workflows. A workflow is a sequence of tasks that need to run in a specific order, such as processing data, running scripts, or coordinating services across different systems. Kestra allows developers to define these workflows as code and execute them reliably at scale.

The platform provides tools for scheduling tasks, monitoring execution, and handling failures when something goes wrong. Because Kestra is open source, developers from around the world can contribute by fixing bugs, improving features, or optimizing performance. In this project, contributors work with a large codebase that includes backend services and APIs that manage workflow definitions and execution.

## Why is this open source project important?

Kestra is important because it helps engineers automate complex workflows and manage large systems more efficiently. Many companies rely on workflows to run tasks like data processing, scheduled jobs, and system integrations. A platform like Kestra allows developers to define these workflows as code, monitor them, and ensure that tasks run reliably without manual intervention.

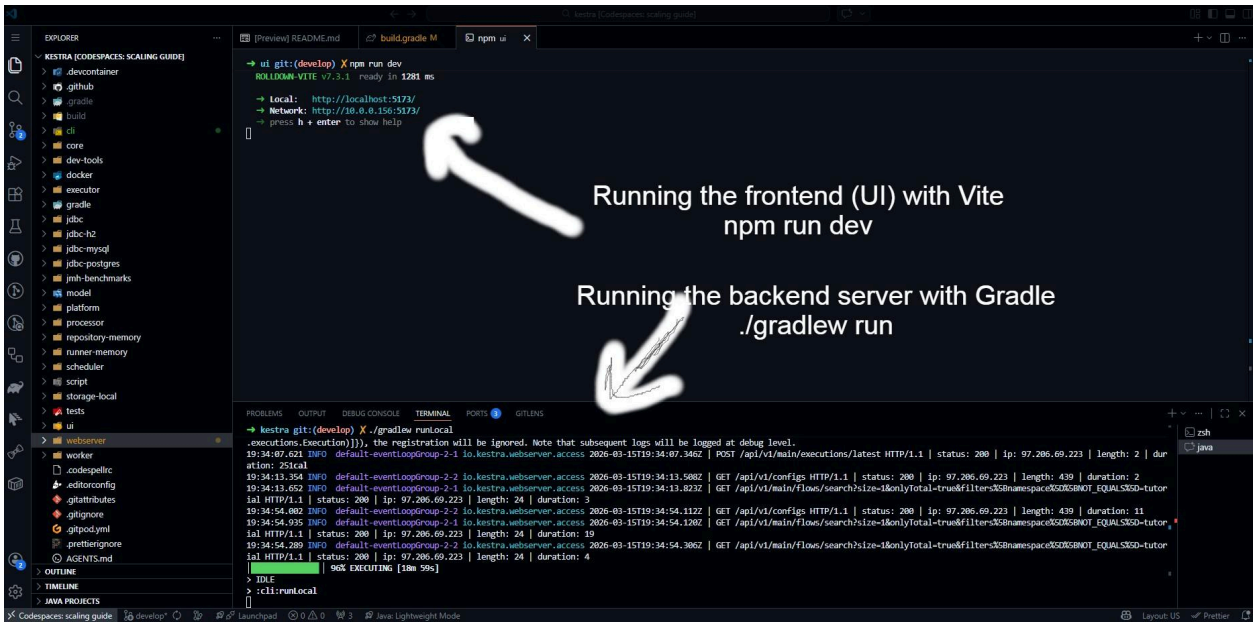
As an open source project, Kestra is also important because it allows developers from around the world to collaborate and improve the platform together. Contributors can fix bugs, add new features, and improve performance, which helps the software grow and become more reliable over time. Open source projects like Kestra also provide learning opportunities for new developers, allowing them to gain real-world experience by working on production-level software used by others.

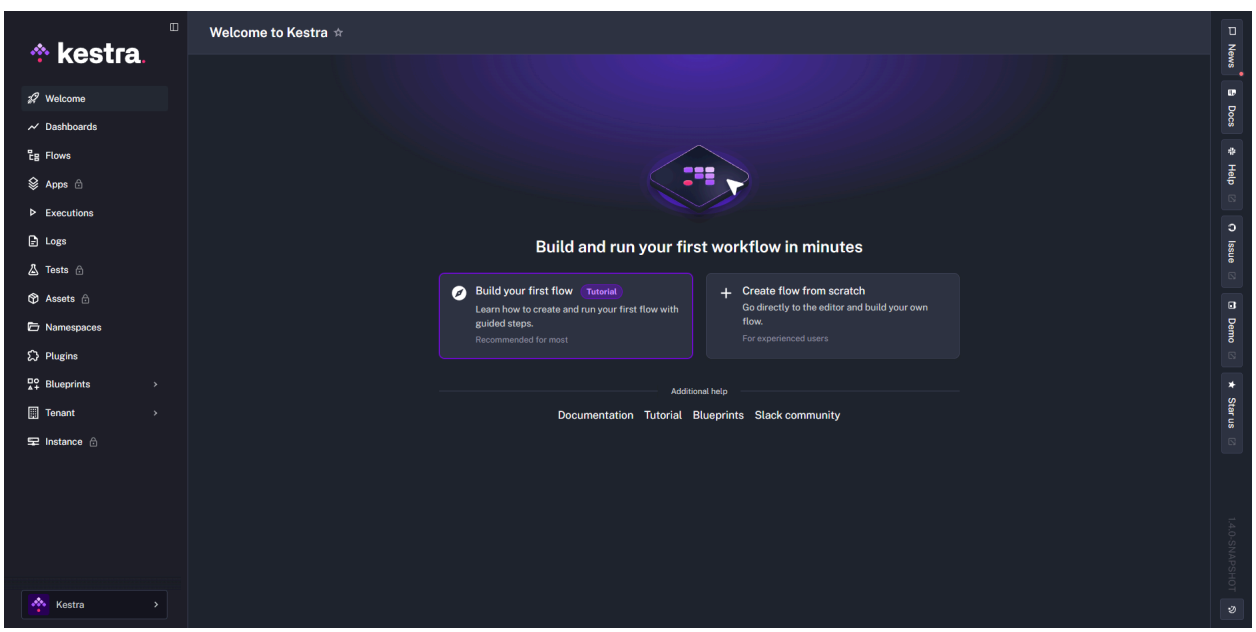
## Who is the typical user of the project and what do they use it for? Give an example scenario where the app could do something useful for someone.

The typical users of Kestra are software engineers, data engineers, and DevOps engineers who need to automate and manage workflows. These users often work with systems that require multiple tasks to run in a specific order, such as processing data, running scripts, or coordinating services across different platforms. Kestra helps them schedule, monitor, and control these tasks so that everything runs automatically and reliably.

For example, a data engineer at a company might use Kestra to automate a daily data pipeline. Every night, the workflow could automatically collect data from different sources, clean and process the data, store it in a database, and generate reports for the team the next morning. Instead of running these tasks manually, Kestra ensures the entire workflow runs automatically and alerts the engineer if something fails.

**screenshot of the app running on your local machine and annotate it with arrows to explain what is happening in the screenshot.**



## What is the issue that you addressed for the project? It's linked to the GitHub issue.

The issue I worked on is "OSS **API Bug: PUT Request to Non-Existent Flow Causes Server Freeze.”**
GitHub link: https://github.com/kestra-io/kestra/issues/12984

This issue happens when a PUT request is sent to update a flow that does not exist. Instead of returning an error, the server freezes or becomes unresponsive

## Why is this issue important to the open source project?

This issue is important because it affects the stability and reliability of the system. A server freeze can disrupt users, waste resources, and make the API unsafe to use in production. Fixing it ensures proper error handling, improves user experience, and prevents crashes in real-world applications.

## Identify the key location(s) of the codebase that are relevant to solving the issue.

**1. `FlowController.java`**

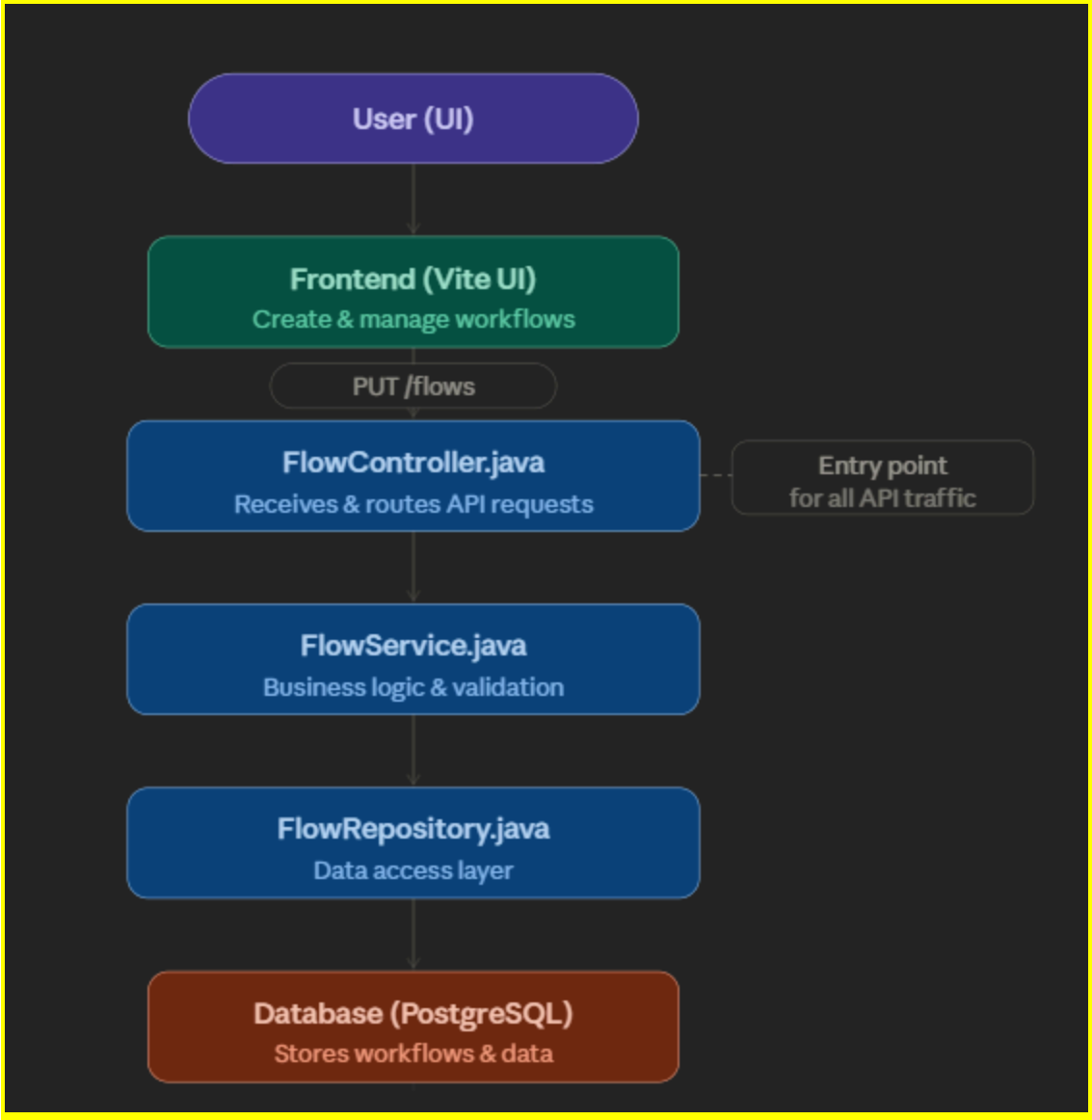
`webserver/src/main/java/io/kestra/webserver/controllers/api/FlowController.java`
This file handles the **API endpoints for flows**, including requests such as GET, POST, PUT, and DELETE. It is relevant to the issue because it is likely where the **PUT request to update a flow is received and processed by the backend**.

## Tech Stack

| Technology | Purpose |
|---|---|

| | |
|---|---|
| Java | Main programming language used to build the backend of Kestra |
| Spring Boot | Framework that handles API requests like GET, PUT, POST, and DELETE |
| REST API | Allows communication between the frontend and backend using HTTP requests |
| Database (PostgreSQL) | Stores workflows (flows) and their data |
| Vite + Frontend UI | Provides the user interface where users can create and manage workflows |
| Gradle | Used to build and run the backend server |

## System Diagram




In this system, the `FlowController.java` file is where the API request first enters the backend. This is important because it is directly related to the issue I was investigating involving PUT requests to flows.

## Workflow (Using the Issue Example)

To understand how the system works, I followed the flow of a PUT request sent to update a workflow.

1. A user sends a PUT request to update a flow, either through the UI or using a tool like curl.
2. The request is sent to the backend API.
3. The request first reaches FlowController.java, which handles incoming API requests.
4. The controller passes the request to FlowService.java, where the main logic is handled.
5. The service attempts to update the flow by interacting with FlowRepository.java.
6. The repository communicates with the database to find and update the flow.

In the case of this issue, the request is sent to a flow that does not exist. Based on the original issue, this should cause the server to freeze. However, during my testing, the system behaved correctly and returned an error instead of freezing.

This shows that the request is properly handled through the system and that the backend correctly identifies when a flow does not exist.

```
zsh
➜ kestra git:(develop) ✗ curl -v -u "jocielosorio55@gmail.com:Nalayyeti_25" -X PUT "http://localhost:8080/api/v1/main/flows/test/freeze-bug" -H "Conten
t-Type: application/x-yaml" --data-binary $'id: freeze-bug\nnamespace: test\ntasks:\n  - id: log\n    type: io.kestra.plugin.core.log.Log\n    message:
Hello'
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
* Server auth using Basic with user 'jocielosorio55@gmail.com'
> PUT /api/v1/main/flows/test/freeze-bug HTTP/1.1
> Host: localhost:8080
> Authorization: Basic am9jaWVsb3NvcmlvNTVAZ21haWwuY29tOk5hbGF5eWV0aV8yNQ==
> User-Agent: curl/8.5.0
> Accept: */*
> Content-Type: application/x-yaml
> Content-Length: 108
>
< HTTP/1.1 404 Not Found
< content-type: application/json
< date: Thu, 2 Apr 2026 16:56:47 GMT
< content-length: 233
<
* Connection #0 to host localhost left intact
{"message":"Not Found","logref":null,"path":null,"_links":{"self":{"href":"/api/v1/main/flows/test/freeze-bug","templated":false,"profile":null,"depreca
tion":null,"title":null,"hreflang":null,"type":null,"name":null}},"_embedded":{}}%
➜ kestra git:(develop) ✗
```

throws 404 error insted on frezzing server

***What is the status of the technical challenge?***

## Reproduction Attempt and Result

As part of my investigation, I attempted to reproduce the issue by sending a PUT request to a non-existent flow using curl. According to the issue, this action should cause the server to freeze. However, in my testing, the server did not freeze. Instead, it returned a **404 Not Found** response.

```
zsh
➜ kestra git:(develop) ✗ curl -v -u "jocielosorio55@gmail.com:Nalayyeti_25" -X PUT "http://localhost:8080/api/v1/main/flows/test/freeze-bug" -H "Conten
t-Type: application/x-yaml" --data-binary $'id: freeze-bug\nnamespace: test\ntasks:\n  - id: log\n    type: io.kestra.plugin.core.log.Log\n    message:
Hello'
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
* Server auth using Basic with user 'jocielosorio55@gmail.com'
> PUT /api/v1/main/flows/test/freeze-bug HTTP/1.1
> Host: localhost:8080
> Authorization: Basic am9jaWVsb3NvcmlvNTVAZ21haWwuY29tOk5hbGF5eWV0aV8yNQ==
> User-Agent: curl/8.5.0
> Accept: */*
> Content-Type: application/x-yaml
> Content-Length: 108
>
< HTTP/1.1 404 Not Found   throws 404 error insted on frezzing server
< content-type: application/json
< date: Thu, 2 Apr 2026 16:56:47 GMT
< content-length: 233
<
* Connection #0 to host localhost left intact
{"message":"Not Found","logref":null,"path":null,"_links":{"self":{"href":"/api/v1/main/flows/test/freeze-bug","templated":false,"profile":null,"depreca
tion":null,"title":null,"hreflang":null,"type":null,"name":null}},"_embedded":{}}%
➜ kestra git:(develop) ✗
```

This result shows that the system is correctly handling the request by identifying that the flow does not exist. The server remained responsive, and no errors or crashes occurred during the process. This suggests that the issue is likely caused by differences in environment configuration rather than a bug in the Kestra codebase.

## Final Solution

The final solution was to investigate and verify whether the reported bug could be reproduced. Instead of changing the code, I tested the system by sending PUT requests to non-existent flows and tracing how the request moved through the backend, starting from **FlowController.java** in the Spring Boot framework.

Through testing, I confirmed that the system correctly returns a **404 Not Found** response when the flow does not exist. This shows that the backend is working as expected and that no code changes were needed. The issue is likely caused by environment differences rather than a bug in the Kestra codebase.

## Testing and Verification

To verify this, I used curl to send a PUT request to a non-existent flow. The server returned a **404 Not Found** response and remained responsive, as shown in Figure 1. This confirms that the issue could not be reproduced and that the system behaves correctly.